

Solving Recurrences Made Easy

The Master Theorem & Exam Preparation

Lecture 26

Data Structures and Algorithms

Why We Need the Master Theorem

In our recent lectures on Sorting and Searching, we encountered **Divide and Conquer** algorithms.

When we analyze their time complexity, we often get **recurrence relations**:

- **Merge Sort:** $T(n) = 2T(n/2) + O(n)$
- **Binary Search:** $T(n) = T(n/2) + O(1)$

Solving these using the Substitution Method or Recursion Tree Method can be tedious and prone to mathematical errors during exams.

The Solution: The **Master Theorem** provides a "cookbook" method to instantly solve a wide class of recurrence relations.

Part 1: The Master Theorem

Concept and Cases

The General Form

The Master Theorem applies to recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

Where:

- n : Size of the input
- $a \geq 1$: Number of subproblems in each recursive step (how many recursive calls)
- $b > 1$: The factor by which the subproblem size is reduced
- $f(n)$: The cost of the work done outside the recursive calls (e.g., dividing the problem and merging the results)

The Core Idea: We compare the cost of splitting/merging, $f(n)$, with the number of leaves in the recursion tree, which is proportional to $n^{\log_b a}$.

The Core Intuition: Root vs. Leaves

To understand the three cases, we calculate a "watershed" value: $n^{\log_b a}$.

Where does $n^{\log_b a}$ come from?

- **Tree Depth:** We divide the problem size n by b at each step. We hit the base case at depth $\log_b n$.
- **Leaf Count:** The tree branches a times per level. The number of leaves at the bottom is $a^{\text{depth}} = a^{\log_b n}$.
- **The Math Trick:** By logarithmic identity, $a^{\log_b n}$ is exactly equal to $n^{\log_b a}$.
- **The Meaning:** Assuming each base case (leaf) takes $O(1)$ time, $n^{\log_b a}$ represents the **total work done at the very bottom level** of the recursion.

Meanwhile, $f(n)$ represents the work done at the **very top level** (the root).

The Master Theorem is simply a battle: **Where is the bottleneck?** Is the algorithm spending most of its time at the root, at the leaves, or is the work spread evenly?

Case 1: The Leaves Dominate (Bottom-Heavy)

Imagine an algorithm where dividing the problem is fast, but you have created so many subproblems that solving the base cases takes forever.

- **The Condition:** $f(n)$ is polynomially *smaller* than $n^{\log_b a}$.
(Mathematically: $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$)
- **What it means:** The vast majority of the algorithm's time is spent at the bottom of the recursion tree. The leaves dominate the total cost.
- **The Solution:** The overall time complexity is simply determined by the work at the leaves!

$$T(n) = \Theta(n^{\log_b a})$$

Case 2: It is a Tie! (Evenly Distributed)

Imagine an algorithm where the time spent merging the results perfectly balances the sheer number of subproblems. (Merge Sort is the classic example).

- **The Condition:** $f(n)$ is proportionally the *same* as $n^{\log_b a}$.
(Mathematically: $f(n) = \Theta(n^{\log_b a} \log^k n)$ for $k \geq 0$)
- **What it means:** The work is spread perfectly evenly across every level of the tree.
- **The Solution:** Since each level does roughly $f(n)$ work, and there are $\log n$ levels, we simply multiply the work by $\log n$!

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

(Most commonly $k = 0$, so the answer is just $f(n) \log n$)

Case 3: The Root Dominates (Top-Heavy)

Imagine an algorithm where you spend an enormous amount of time dividing the data or merging it, but the subproblems themselves shrink extremely fast.

- **The Condition:** $f(n)$ is polynomially *larger* than $n^{\log_b a}$.
(Mathematically: $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$)
- **What it means:** The vast majority of the algorithm's time is spent at the very first step (the root). The deeper levels take increasingly negligible time.
- **The Solution:** Because the root heavily dominates, the overall time complexity is simply the work done at the root!

$$T(n) = \Theta(f(n))$$

Part 2: Solving Problems with the Master Theorem

Step-by-Step Examples

Problem 1: Merge Sort

Recurrence: $T(n) = 2T(n/2) + n$

Step 1: Identify constants

- $a = 2$ (two subproblems)
- $b = 2$ (each half the size)
- $f(n) = n$ (linear time to merge)

Step 2: Calculate $n^{\log_b a}$

- $\log_b a = \log_2 2 = 1$
- So, $n^{\log_b a} = n^1 = n$

Step 3: Compare and Apply

- $f(n) = n$ and $n^{\log_b a} = n$. They are equal!
- This is **Case 2** (with $k = 0$).
- **Solution:** $T(n) = \Theta(n \log n)$

Problem 2: Binary Search

Recurrence: $T(n) = T(n/2) + 1$

Step 1: Identify constants

- $a = 1$ (one subproblem)
- $b = 2$ (search space halved)
- $f(n) = 1$ (constant time to find mid and compare)

Step 2: Calculate $n^{\log_b a}$

- $\log_b a = \log_2 1 = 0$
- So, $n^{\log_b a} = n^0 = 1$

Step 3: Compare and Apply

- $f(n) = 1$ and $n^{\log_b a} = 1$. They are equal!
- This is **Case 2** (with $k = 0$).
- **Solution:** $T(n) = \Theta(\log n)$

Problem 3: A Classic Exam Problem

Recurrence: $T(n) = 9T(n/3) + n$

Step 1: Identify constants

- $a = 9$
- $b = 3$
- $f(n) = n$

Step 2: Calculate $n^{\log_b a}$

- $\log_b a = \log_3 9 = 2$
- So, $n^{\log_b a} = n^2$

Step 3: Compare and Apply

- $f(n) = n$ is polynomially smaller than n^2 (i.e., $n = O(n^{2-\epsilon})$ for $\epsilon = 1$).
- The work at the leaves dominates. This is **Case 1**.
- **Solution:** $T(n) = \Theta(n^2)$

Problem 4: The Heavy Root

Recurrence: $T(n) = 3T(n/4) + n \log n$

Step 1: Identify constants

- $a = 3$
- $b = 4$
- $f(n) = n \log n$

Step 2: Calculate $n^{\log_b a}$

- $\log_b a = \log_4 3 \approx 0.793$
- So, $n^{\log_b a} = n^{0.793}$

Step 3: Compare and Apply

- $f(n) = n \log n$ is polynomially strictly larger than $n^{0.793}$.
- The work at the root dominates. This is **Case 3**.
- **Solution:** $T(n) = \Theta(n \log n)$

Part 3: Exam Preparation

Mathematical Problems on Sorting & Searching

Sorting Comparisons

Question 1:

Suppose you are given an array of n elements that is already sorted in reverse order. You want to sort it in ascending order. Exactly how many swaps and comparisons will **Selection Sort** and **Bubble Sort** perform?

Solution Strategy:

- **Selection Sort:**
 - **Comparisons:** Always $\frac{n(n-1)}{2}$, regardless of input.
 - **Swaps:** Exactly $n - 1$ swaps (or less if optimized to not swap an element with itself, but standard is $n - 1$).
- **Bubble Sort:**
 - **Comparisons:** Since it's in reverse order, it will do all passes. Total comparisons = $\frac{n(n-1)}{2}$.
 - **Swaps:** Every comparison results in a swap because every pair is out of order. Total swaps = $\frac{n(n-1)}{2}$.

Takeaway: For reverse-sorted data, Bubble Sort does $O(n^2)$ swaps, while Selection Sort does $O(n)$ swaps. Selection sort minimizes memory writes!

Lower Bounds

Question 2:

Using the decision tree model, prove that any comparison-based sorting algorithm requires at least $\Omega(n \log n)$ comparisons to sort an array of n elements. What is the minimum height of a decision tree for sorting 5 elements?

Solution Strategy:

1. **Proof Idea:** A decision tree for sorting n elements must have at least $n!$ leaves (one for each possible permutation).
2. A binary tree of height h has at most 2^h leaves.
3. Therefore, $2^h \geq n! \implies h \geq \log_2(n!)$.
4. By Stirling's approximation, $\log_2(n!) = \Omega(n \log n)$. Thus, $h = \Omega(n \log n)$.

For $n = 5$:

- $5! = 120$.
- We need $2^h \geq 120$.
- Since $2^6 = 64$ and $2^7 = 128$, the minimum height h is **7**.
- It takes at least 7 comparisons in the worst case to sort 5 elements.

Binary Search Trees vs Array Search

Question 3:

You have $n = 1023$ elements.

- (a) What is the maximum number of comparisons required to search for an element using Binary Search on a sorted array?
- (b) If the elements are inserted into a Binary Search Tree (BST) in perfectly balanced order, what is the height of the tree?

Solution Strategy:

- **(a) Binary Search Array:**
 - Maximum comparisons = $\lfloor \log_2 n \rfloor + 1$ (or $\lceil \log_2(n + 1) \rceil$).
 - For $n = 1023$, $\log_2(1024) = 10$.
 - Maximum comparisons = **10**.
- **(b) Balanced BST:**
 - A perfectly balanced BST with n nodes has height $h = \lfloor \log_2 n \rfloor$.
 - For $n = 1023$, the tree has 1023 nodes.
 - Height = $\lfloor \log_2(1023) \rfloor = 9$.
 - (Note: Nodes at height 0 to 9 means 10 levels, which perfectly matches the 10 comparisons in array binary search!).

Summary & Exam Tips

- **Master Theorem** is your quickest tool for analyzing divide-and-conquer algorithms. Always check if the recurrence fits $T(n) = aT(n/b) + f(n)$.
- Remember the **Three Cases**: Leaves dominate (Case 1), Even split (Case 2), Root dominates (Case 3).
- **Sorting Algorithms**: Don't just memorize complexities. Understand *why* an algorithm is $O(n^2)$ or $O(n \log n)$ by tracing the nested loops or recursion.
- **Searching**: Binary search is $O(\log n)$ but requires the overhead of sorting $O(n \log n)$ first. Use it when searching happens frequently on static data.

Good luck with your exam preparations!